

1.1.1 USE CASE – Digital Twin for Mobility: Climate-Resilient Highway Management Through Fire Risk Forecasting

Across Europe, and especially in Southern regions like Portugal, climate change is driving more frequent and severe rural fires that increasingly disrupt mobility networks and threaten critical infrastructure such as highways. Ensuring continuity of transport and emergency operations requires shifting from reactive responses to predictive, data-driven management. A Digital Twin for Climate-Resilient Highways integrates geospatial intelligence, climate analytics, risk modelling, real-time data and high-resolution simulations to forecast wildfire behavior and anticipate its impact on highway corridors. This use case aims to strengthen resilience, support informed decision-making and safeguard mobility flows at national and European scales.

In this context, this document proposes a technological architecture entirely based on open-source software for the simulation of rural fire propagation, integrating physical modelling, spatial data management, and dynamic ingestion of meteorological data. The proposed solution is centred on the ForeFire simulation engine, complemented by a data infrastructure built on PostgreSQL with the PostGIS extension, and a Python-based integration module for process orchestration.

Among the available open-source tools for fire simulation, ForeFire and Cell2Fire stand out. Although both support modelling fire spread, they differ significantly in their conceptual approach, physical assumptions, and ease of operational integration.

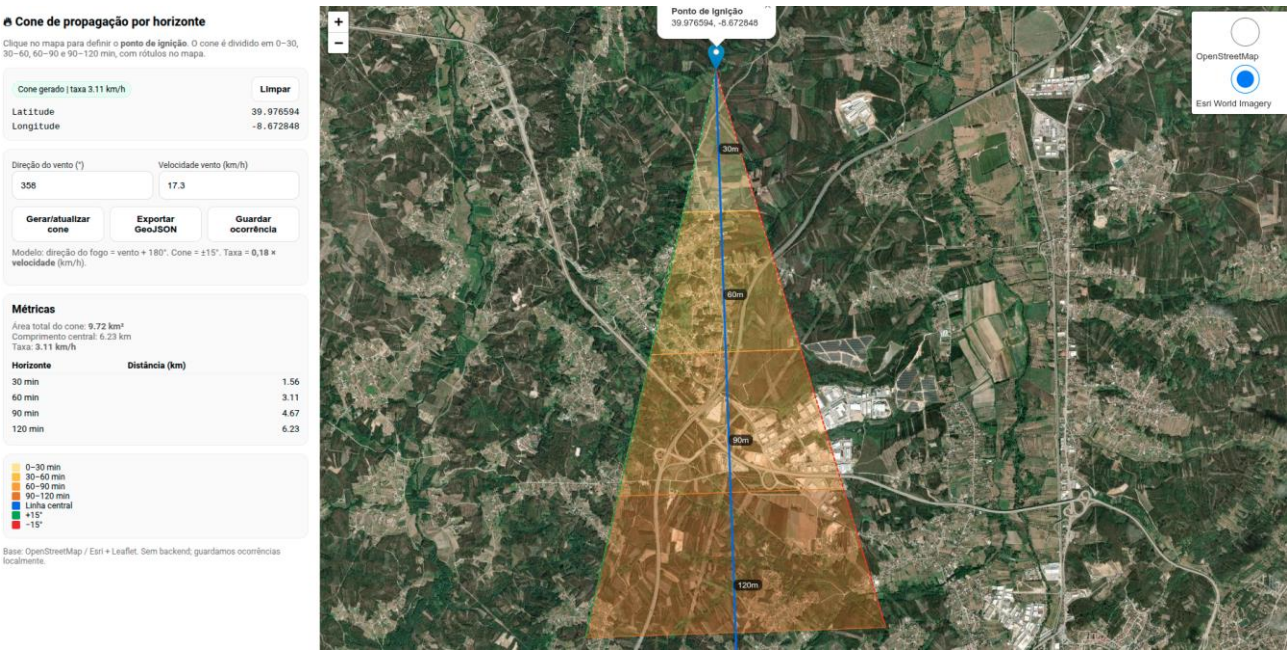
Cell2Fire is based on a cell-based model, where the territory is represented as a regular grid and each cell evolves according to probabilistic rules dependent on the state of neighbouring cells. This approach is particularly well suited for stochastic analyses and forest-planning risk assessment, enabling multiple simulations with parameter variation. However, its integration with dynamic data flows—such as real-time observed meteorology—is less straightforward.

ForeFire, on the other hand, implements a semi-empirical propagation model based on Rothermel's equations, widely used in surface fire modelling. Fire behaviour is described through the Rate of Spread (ROS), intensity, and the combined influence of wind, slope, and fuel properties. This physical foundation provides greater robustness when aiming to approximate real-world conditions and integrate observed meteorological data.

From a technological perspective, ForeFire features a modular structure based on standardized input files (fuels.csv, landscape.nc, and the simulation script, ForeFire Script File (.ff)), which facilitates automation and integration with external services. This characteristic is essential for an operational system that needs to generate scenarios based on variable inputs.

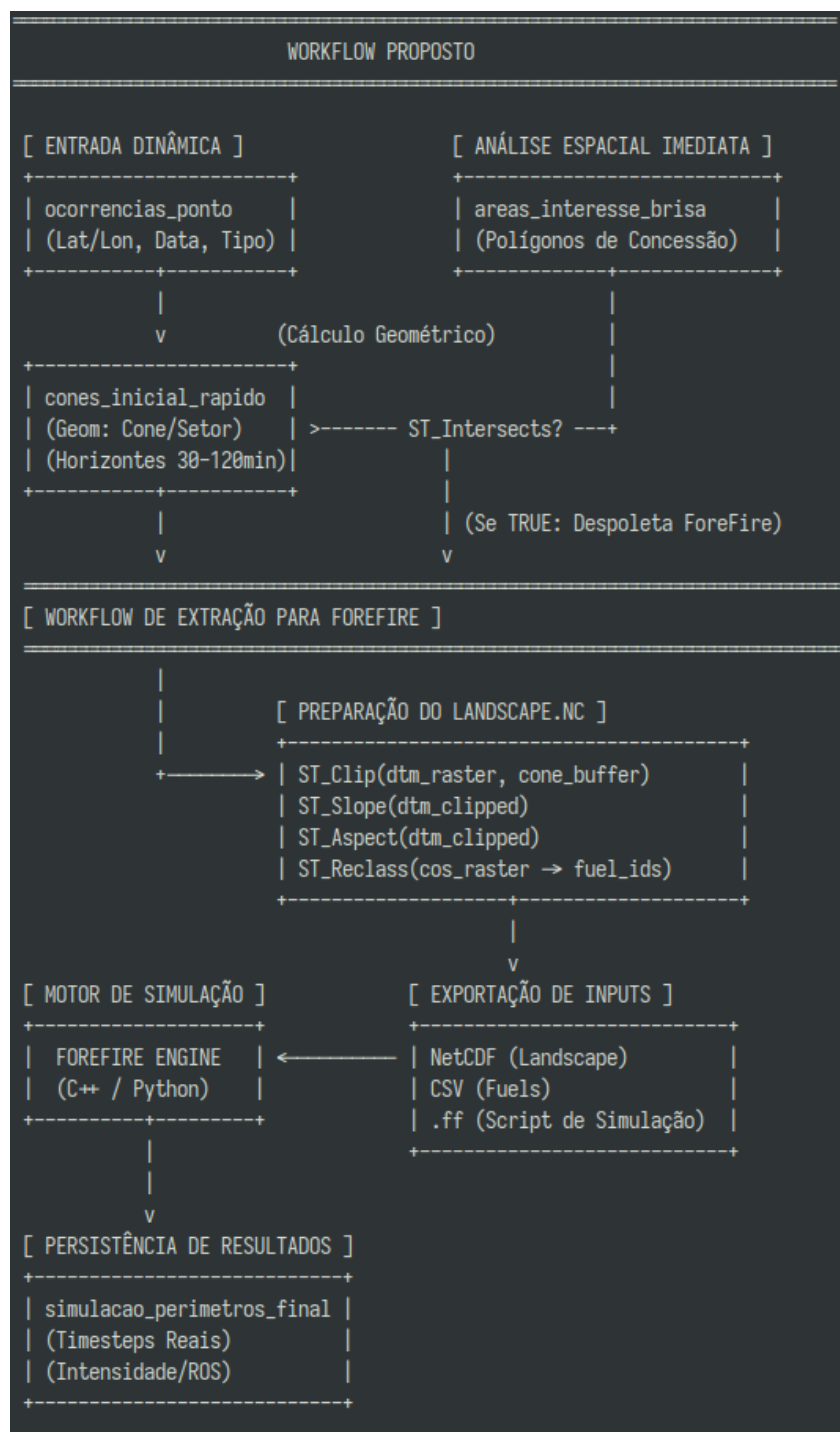
1.1.1.1 Proposed Workflow

The proposed workflow includes two stages. After an incident is received, it is recorded in a simplified application that automatically generates a propagation cone for a given time horizon, as illustrated in the image below.



If this point, or its propagation cone, intersects the area of interest for Brisa’s operations—and considering that both the initial point and its propagation cone are stored in the PostGIS database—the next process is triggered. This being a subsequent process consisting of a more detailed analysis using a fire-spread model, in this case, the ForeFire engine.

The operational workflow should be implemented following a structure similar to the diagram presented below.



1.1.1.2 Data Sources

To enable accurate and operationally meaningful wildfire-spread simulations, it is essential to incorporate meteorological variability into the modelling workflow. Wind conditions play a decisive role in determining the rate, direction and intensity of fire propagation, making their continuous acquisition, processing and integration a critical component of the system. The following steps outline the proposed method for collecting, transforming, and injecting dynamic wind data into the ForeFire simulation engine.

- **Data Collection (Python):** Use the endpoint <https://api.ipma.pt/open-data/observation/meteorology/stations/observations.json> to obtain real-time wind speed (ff) and wind direction (dd);

Example of the data

```
{
  "geometry": {
    "type": "Point",
    "coordinates": [
      -8.44,
      40.4389
    ]
  },
  "type": "Feature",
  "properties": {
    "intensidadeVentoKM": 10.4,
    "temperatura": 16.9,
    "idEstacao": 1210705,
    "pressao": -99.0,
    "humidade": 63.0,
    "localEstacao": "Anadia",
    "precAcumulada": 0.0,
    "idDireccVento": 9,
    "radiacao": 493.7,
    "time": "2026-03-15T17:00:00",
    "intensidadeVento": 2.9,
    "descDirVento": "N"
  }
},
{
  "geometry": {
    "type": "Point",
    "coordinates": [
      -8.4552,
      40.2135
    ]
  },
  "type": "Feature",
  "properties": {
    "intensidadeVentoKM": 11.2,
    "temperatura": 17.8,
    "idEstacao": 1210707,
    "pressao": -99.0,
    "humidade": 67.0,
    "localEstacao": "Coimbra, Bencanta",
    "precAcumulada": 0.0,
    "idDireccVento": 8,
    "radiacao": 1399.2,
    "time": "2026-03-15T17:00:00",
    "intensidadeVento": 3.1,
```

```

        "descDirVento": "NW"
    }
},

```

- Spatial Interpolation: Since IPMA stations are discrete points and ForeFire requires a full wind field over the entire landscape, an interpolation method must be applied:
 - IDW (Inverse Distance Weighting): Simpler and faster;
 - Kriging: More complex and computationally demanding.
- Injection into ForeFire:
 - Via Script (.ff): Using the commands set wind_speed X and set wind_direction Y for a uniform wind field;
 - Via NetCDF: Creating dynamic layers wind_u and wind_v inside landscape.nc for spatially varying wind conditions.

As an example, the Python script for using the IPMA API

```

import requests, psycopg2, math

# 1. Get the ignition location (PostGIS)
conn = psycopg2.connect("dbname=brisa_fire user=postgres")
cur = conn.cursor()
cur.execute("""
    SELECT ST_X(geom), ST_Y(geom)
    FROM ocorrencias_ponto
    ORDER BY id DESC
    LIMIT 1
""")
lon_fire, lat_fire = cur.fetchone()

# 2. Query IPMA (Retrieves meteorology and station coordinates in a single step)
data = requests.get(
    "https://api.ipma.pt/open-data/observation/meteorology/stations/observations.json"
).json()
obs_ipma = data[list(data.keys())[-1]] # Data for the most recent timestamp

# 3. Filter the 3 nearest stations and convert to vectors (U, V)
# Mapping: Direction Class (1-9) -> Degrees
directions = {1: 0, 9: 0, 2: 45, 3: 90, 4: 135, 5: 180, 6: 225, 7: 270, 8: 315}
station_list = []

for id_st, obs in obs_ipma.items():
    if obs and obs['lat'] != -990:
        # Simple distance and wind decomposition (m/s)
        dist = ((obs['lat'] - lat_fire)**2 + (obs['lon'] - lon_fire)**2)**0.5
        rad = math.radians(directions.get(obs['idDireccVento'], 0))

        u = obs['intensidadeVento'] * math.sin(rad) # East-West component
        v = obs['intensidadeVento'] * math.cos(rad) # North-South component

    station_list.append((u, v, obs['lon'], obs['lat'], dist))

```

```
# Select the 3 with the shortest distance
top3 = sorted(station_list, key=lambda x: x[4])[:3]

# 4. Insert into PostGIS (Preparation for IDW interpolation)
cur.execute("TRUNCATE TABLE estacoes_meteorologia_temp")
for u, v, lon, lat, d in top3:
    cur.execute("""
        INSERT INTO estacoes_meteorologia_temp (u, v, geom)
        VALUES (%s, %s, ST_SetSRID(ST_MakePoint(%s, %s), 4326))
        """, (u, v, lon, lat))

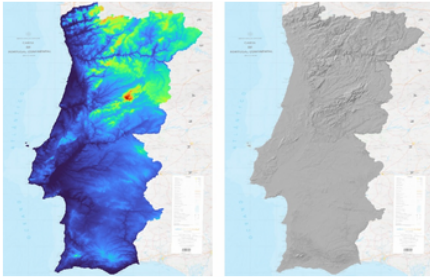
conn.commit()
```

Example of IDW interpolation in PostGIS

```
-- Example of IDW interpolation for the 'U' wind component
-- Repeat the process for the 'V' component
SELECT ST_InterpolateRaster(
  ST_Collect(
    ARRAY[
      ST_MakePoint(-8.5, 40.1, 2.45), -- Station 1 (Lon, Lat, U_Value)
      ST_MakePoint(-8.1, 40.4, 1.80), -- Station 2
      ST_MakePoint(-8.7, 39.9, 3.10)  -- Station 3
    ]
  ),
  'invdist:power:2', -- IDW (Inverse Distance Weighting)
  ST_AddBand(
    ST_MakeEmptyRaster(200, 200, -8.8, 40.6, 0.001),
    '32BF'
  )
) AS wind_field_u;
```

To prepare the DEM for integration into the landscape.nc file, the LiDAR-derived dataset must first be clipped to the area of interest and converted into a uniform grid compatible with the resolution required by the simulation model. Once the DEM has been processed, it can be exported to NetCDF format, ensuring that elevation values are stored in a structured raster band with consistent georeferencing parameters. The DEM used in this workflow is the one derived from DGT's LiDAR, with a spatial resolution of 10 meters and made available by CIIMAR (<https://www.fc.up.pt/pessoas/jagoncal/dtm-lidar-dgt/>). This NetCDF file then serves as the topographic foundation of the ForeFire landscape, supporting the calculation of slope and aspect, which are essential physical drivers of fire spread within the model.

MDT LIDAR DGT com resolução de 10 m para Portugal continental



Este ficheiro MDT foi gerado a partir dos rasters com resolução de 2 metros obtidos no levantamento LIDAR da Direção Geral do Território (DGT), disponíveis em novembro de 2025 5x5 pixels, tendo o ficheiro 28600 colunas por 57800 linhas. As altitudes são números reais (Float32) e referem-se ao datum altimétrico nacional. O sistema de coordenadas é ETRS programs como QGIS ou ArcGIS.

Os dados originais podem ser obtidos no [Centro de Dados da DGT](#). A utilização deste ficheiro deve fazer referência aos dados originais e ao processo de geração deste modelo.

Download

- ↓ MDT, GSD=10 m (3.9 Gb) ←
- ↓ Shaded relief (825 Mb)

Data and web page prepared by: José Alberto Gonçalves ([jagoncal at fc.up.pt](#)) and André Pinhal ([apinhal at fc.up.pt](#))

As mentioned before, ForeFire requires Elevation, Slope, and Aspect. The latter two are calculated in PostGIS (using ST_Slope and ST_Aspect) from the DEM.

Example SQL command

```
-- Extract the terrain subset for the ignition area
SELECT ST_AsGDALRaster(
  ST_Union(rast),
  'NetCDF',
  ARRAY['COMPRESS=DEFLATE', 'FORMAT=NC4']
)
FROM dgt_cos_raster
WHERE ST_Intersects(
  rast,
  ST_Buffer(
    ST_SetSRID(ST_Point(-8.5, 40.2), 4326),
    0.1
  )
);
```


One of the key data and main challenge, in the Portuguese context, when using this type of simulator is related to fuel mapping and to translating the Land Use and Land Cover Map (COS) from DGT into the physical-chemical parameters required by Rothermel-based fuel models."

In practice, this difficulty arises because the COS classification was not originally designed for fire-behaviour modelling; it describes land-use categories rather than fuel structures. Rothermel's equations, however, require detailed parameters such as fuel load, surface-area-to-volume ratio, heat content, dead and live fuel moisture categories, and fuel bed depth. As a result, developing a reliable fuel-model layer requires establishing a robust crosswalk between COS classes and standard fuel models.

Example of Fuel Mapping in Portugal

	model	fuel load (t/ha)	surface-to-volume (1/m)	moisture extinction	Technical Description
Pasto_Seco	rothermel	1.5	3500	0.12	Dry grasses and fine pasture fuels. Very fast spread.
Mato_Denso	rothermel	13.0	2000	0.20	Dense shrubs such as <i>giestas</i> and <i>urzes</i> . High heat release.
Pinhal_Bravo	rothermel	7.0	1500	0.25	Maritime pine forest with needle litter. Moderate surface fire.
Eucaliptal_Limpo	rothermel	12.0	4500	0.25	Managed eucalyptus stands with clean understory.
Eucaliptal_Abandonado	rothermel	25.0	5500	0.20	Critical: high biomass load, loose bark and accumulated shrub layer.
Sobreiral_Alcor	rothermel	5.0	1800	0.30	Cork oak / holm oak woodlands. Slower and more controllable spread.

Adapted from the following bibliographic sources:

- Physical Parameters: Adapted from Fernandes, P. M. (2001). "Classification and characterization of fuel complexes in Portugal."
- Propagation Models: Implementation based on the original Rothermel models (1972), validated for Mediterranean ecosystems by ADAI (Associação para o Desenvolvimento da Aerodinâmica Industrial), University of Coimbra.
- Land-Cover Mapping: Nomenclature based on the Land Use and Land Cover Map (COS 2018/2021) from the Portuguese Directorate-General for Territory (DGT).

So, by reclassifying the COS (Land Use and Land Cover Map), you should use an SQL-based mapping logic:

```
-- Example of mapping in the extraction query:

-- COS code 322 (Eucalyptus stands) → Map to ID 11 if there is shrub layer or 10 if it is clean

SELECT ST_Reclass(rast, 1, '322:11, 311:9, 211:1', '8BUI', 0)
FROM dgt_cos_raster;
```

This table will be read by the loadFuels fuels.csv command inside your ForeFire script (.ff), ensuring that the simulation engine knows exactly how “aggressive” the fire should be when entering a eucalyptus stand.

To operationalize the creation of the landscape.nc file required by ForeFire, the workflow involves extracting each spatial variable directly from PostGIS, converting the resulting rasters into in-memory datasets, and assembling them into a unified NetCDF structure. This approach ensures that elevation, slope, aspect, and fuel type layers are consistently aligned, share the same spatial resolution, and conform to the input specifications expected by the simulation engine. The following Python example illustrates how these layers can be queried, loaded, and combined into a single NetCDF file suitable for fire-spread modelling.

Example of creating the landscape.nc file in Python

```
import psycopg2
import xarray as xr

# 1. Logic to extract from PostGIS and convert to an array
def get_raster_layer(cursor, sql_query):
    cursor.execute(sql_query)
    blob = cursor.fetchone()[0]
    # Temporarily save or read in-memory via GDAL
    return xr.open_dataset(blob)

# 2. Create the unified landscape.nc
# Extract each layer via SQL (as shown in the examples above)
ds_elev = get_raster_layer(cur, "SELECT ST_AsGDALRaster(ST_Union(rast) ... FROM mdt ...)")
ds_slope = get_raster_layer(cur, "SELECT ST_AsGDALRaster(ST_Slope(...) ...)")
ds_fuel = get_raster_layer(cur, "SELECT ST_AsGDALRaster(ST_Union(rast) ... FROM cos_reclass ...)")
ds_aspect = get_raster_layer(cur, "SELECT ST_AsGDALRaster(ST_Aspect(...) ...)")

# 3. Combine the variables into a single NetCDF compatible with ForeFire
landscape = xr.Dataset({
    'elevation': ds_elev.band1,
    'slope': ds_slope.band1,
    'aspect': ds_aspect.band1,
    'fuelType': ds_fuel.band1
```

```
} )
```

```
landscape.to_netcdf("landscape.nc")
```

1.1.1.3 Proposed Software Architecture

For this Use Case to be fully operational, the architecture should include the following components:

- Database (PostGIS): Repository for maps and wildfire historical data.
- Ingestion Layer (Python): Script that queries the IPMA API and prepares the landscape.nc file.
- Simulation Engine (ForeFire): Executes the intensive fire-spread calculations.
- Visual Interface (the one already provided with ForeFire, or QGIS, or Leaflet via API): Where the end user visualizes the fire propagation “moving” across the map.

Simplified Functional Diagram

IPMA → Python → PostGIS → Input Generation → ForeFire → PostGIS → Visualization

More complete diagram

